

Asunto: Modelo Final: Instructivo para usar el modelo pre-entrenado de clasificación de plantas
Fecha: sábado, 24 de mayo de 2025, 3:33:26 p. m. hora estándar de Colombia
De: María Fernanda Izquierdo Aparicio
A: Cristian Leandro Lopez Usa, Néstor Armando Nova Arévalo, Juan Erasmo Gomez Morantes
CC: Ana María Ochoa Muñoz
Datos: image001.png, image002.png, ejemplo.JPG, image004.png, image005.png,
adjuntos: index_to_class_modelo_16_Unet_PlantaEstado_Balanceado_Sin_Data_Aug_100perc.json, Instructivo para cargar el modelo preentrenado Unet.ipynb

Buenas tardes Cristian,

Esperamos que este correo te encuentre bien.

Te escribimos para proporcionarte una guía sobre cómo utilizar el modelo final pre-entrenado de clasificación de imágenes de plantas que hemos desarrollado como parte de nuestro proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana.

Descripción del Modelo

Este modelo está diseñado para clasificar imágenes de 9 especies de plantas alimenticias en función de su especie y estado (saludable o enferma). En total clasifica 18 categorías (9 plantas entre saludables y enfermas), y se basa en una arquitectura UNET.

Todo este modelo fue desarrollado usando el siguiente código, te lo compartimos por si te interesa conocer como fue entrenado el modelo:

[Modelo 16 - Unet_Planta_Estado Balanceado Sin Data Augmentation 100%.ipynb](#)

Para que tengas en cuenta este es el modelo final de nuestro proyecto de grado ya que pudimos obtener las siguientes métricas:

- **Train accuracy: 94.3%**
→ El modelo acierta en el 94.3% de las imágenes del conjunto de entrenamiento. Significa que aprendió correctamente de los datos que vio durante la fase de train.
- **Validation accuracy: 92.4%**
→ El modelo acierta en el 92.4% de los datos usados para validar el aprendizaje (que no vio durante el entrenamiento). Es una buena señal de que no está sobreajustado.
- **Test accuracy: 94.18%**
El modelo acierta en el 94.18% de las imágenes completamente nuevas (conjunto de prueba). Esto confirma que generaliza correctamente a nuevos datos, lo cual es un factor importante para usarlo en la app de “Residuos Conciencia”.

En el archivo podrás ver todos los resultados que comprueban estas conclusiones.

Archivos necesarios para cargar el modelo:

- Archivo con modelo pre-entrenado: [modelo_16_Unet_PlantaEstado_Balanceado_Sin_Data_Aug_100perc.1.h5](#) (link)
- Revisar adjuntos

Instrucciones de Uso

A continuación, te detallo los pasos necesarios para cargar y utilizar el modelo. Ya está todo codificado en el archivo llamado “[Instructivo para cargar el modelo preentrenado Unet.ipynb](#)” adjunto pero recomendamos leer los pasos que dejamos abajo:

1. Instalación de Dependencias

- Asegúrate de tener instalado TensorFlow. Puedes instalarlo utilizando el siguiente comando

Bash



```
pip install tensorflow
```

2. Cargar el Modelo .h5

- El modelo pre-entrenado (archivo .h5 anexo) se guarda en un archivo con extensión .h5. Para cargarlo, utiliza el siguiente código en Python.

Python



```
from tensorflow.keras.models import load_model

# Carga el modelo
model = load_model('ruta_al_archivo_del_modelo.h5')

# Imprime la arquitectura del modelo para verificar la carga
model.summary()
```

- **Archivos Necesarios:**
 - Debes tener el archivo del modelo pre-entrenado (.h5) compartido en el link del inicio del correo. Asegúrate de que la ruta al archivo en el código Python sea correcta.
- **Qué cambiar en el código:**
 - Reemplaza 'ruta_al_archivo_del_modelo.h5' con la ruta real de donde guardaste el archivo del modelo .h5.

3. Preprocesamiento de la Imagen

- Antes de clasificar una imagen, es necesario preprocesarla para que tenga las dimensiones y el formato esperado por el modelo. Utiliza el siguiente código:

Python



```
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.resnet50 import preprocess_input
import numpy as np

# Carga la imagen
img_path = 'ruta_a_la_imagen.jpg'
img = image.load_img(img_path, target_size=(150, 150)) # Ajusta el tamaño
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_preprocessed = preprocess_input(img_array)
```

- **Qué cambiar en el código:**
 - Reemplaza 'ruta_a_la_imagen.jpg' con la ruta a la imagen que deseas clasificar. Adjuntamos una imagen de ejemplo por si te sirve.
 - Asegúrate de que target_size coincida con el tamaño de entrada esperado por el modelo (en este caso, 150x150).
 -

4. Realizar la Predicción

- Utiliza el modelo cargado para predecir la clase de la imagen preprocesada:

Python



```
predictions = model.predict(img_preprocessed)
predicted_class_index = np.argmax(predictions)
```

Decodificar la Predicción

- Para obtener el nombre de la clase predicha, necesitar utilizar un diccionario de mapeo de índices a nombres de clases que se encuentra adjunto a este correo
(index_to_class_modelo_16_Unet_PlantaEstado_Balanceado_Sin_Data_Aug_100perc.json)

Esperamos que esta guía te sea de utilidad. Si tienes alguna pregunta o necesitas ayuda adicional, no dudes en contactarnos.

Cordialmente,

María Fernanda Izquierdo Aparicio

Estudiante Maestría Analítica para la
Inteligencia de Negocios

mariaizquierdo@javeriana.edu.co

Cel: 310 270 7900

Ana María Ochoa Muñoz

Estudiante Maestría Analítica para la
Inteligencia de Negocios

an.ochoa@javeriana.edu.co

Cel: 312 457 3399